

Kyrozen 项目方案

项目名称：Kyrozen

项目类型：AI 产品发现与开发 Agent

基础框架：OpenKyrozen

核心定位：帮助普通人从发现问题开始，完成软件、硬件或软硬件结合产品的规划、开发、验证与持续迭代。

一、项目概述

Kyrozen 是一个面向个人创造者、学生、创业者和小型团队的 AI 产品开发 Agent。

它的目标不是简单地回答产品问题，也不是只帮助用户写代码，而是承担一个完整产品团队的职责，陪伴用户完成从问题发现到产品落地的全过程。

用户不需要提前知道：

- 应该开发网站还是 App
- 是否需要使用硬件
- 应该选择什么技术
- 应该购买哪些电子元件
- 如何编程
- 如何设计产品
- 如何进行市场调研
- 如何验证产品是否有价值

用户可以从一句模糊的描述开始：

“我跑步时经常觉得音乐节奏和运动状态不匹配。”

Kyrozen 将帮助用户逐步完成：

发现问题

- 定义问题
- 验证问题
- 市场和竞品调研
- 判断产品机会
- 设计解决方案
- 确定 MVP
- 选择软件、硬件或混合路线
- 开发产品
- 测试产品

- 收集反馈
- 持续优化

Kyrozen 的最终目标是：

让一个没有产品、编程或工程背景的人，也能像拥有一支完整研发团队一样，把问题和想法转化为真实产品。

二、产品愿景

过去，开发一个完整产品通常需要：

- 用户研究人员
- 市场研究人员
- 产品经理
- UI/UX 设计师
- 软件工程师
- 嵌入式工程师
- 电子工程师
- 机械设计师
- 采购人员
- 测试工程师
- 项目经理

个人通常只能拥有其中一部分能力。

现有 AI 工具虽然可以帮助用户生成文档、代码、界面或硬件方案，但通常只覆盖产品开发中的某一个环节。

Kyrozen 的愿景是将这些能力整合成一个长期协作的 AI 产品团队。

它不仅帮助用户执行已经想好的方案，还应该在方案形成之前判断：

- 这个问题是否真实存在
- 是否值得解决
- 是否已经有成熟产品
- 现有产品为什么没有解决好
- 产品应该是软件、硬件还是两者结合
- 哪种方案最容易验证核心价值
- 是否有必要继续开发

因此，Kyrozen 不默认每个想法都是好想法。

当研究结果表明项目价值不高、竞争过强、技术不可行，或者存在更简单的解决方法时，Kyrozen 应明确告诉用户，而不是一味鼓励继续开发。

三、Kyrozen 与 OpenKyrozen 的关系

3.1 OpenKyrozen

OpenKyrozen 是一个通用、开源、自学习的 AI Agent 框架。

它提供：

- 多模型支持
- 工具调用
- 文件操作
- 命令执行
- Git 管理
- 网络搜索
- 长期记忆
- 任务规划
- 插件系统
- Web API
- 自学习机制

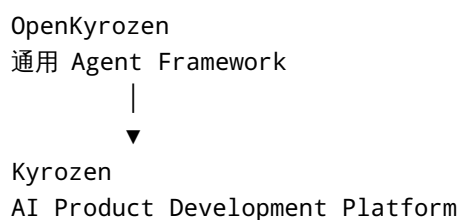
OpenKyrozen 可以继续作为独立开源项目存在，面向需要通用 Agent 能力的开发者。

3.2 Kyrozen

Kyrozen 是建立在 OpenKyrozen 能力之上的正式产品。

它不只是一个通用 Agent，而是拥有明确产品开发目标、流程和界面的垂直 Agent。

两者关系可以理解为：



OpenKyrozen负责提供底层能力。

Kyrozen负责提供：

- 产品开发流程
 - 专业产品 Agent
 - 项目管理
 - 市场研究
 - 软件开发
 - 硬件开发
 - 采购
 - 测试
 - 项目自学习
 - 面向普通用户的产品体验
-

四、目标用户

4.1 完全没有技术背景的创造者

这类用户可能有现实问题或产品想法，但不了解：

- 产品设计
- 编程
- 软件架构
- 电子电路
- 嵌入式系统
- 市场分析

Kyrozen需要使用普通语言与用户交流，并把复杂开发流程转换成可理解的步骤。

4.2 学生和 Maker

包括：

- 中学生
- 大学生
- 科创项目参与者
- Arduino 用户
- 机器人社团
- 创客
- 编程爱好者
- STEM 竞赛团队

Kyrozen不仅帮助他们完成项目，也帮助他们理解为什么采用某种方案。

4.3 独立开发者

独立开发者可能擅长软件，但不了解：

- 市场研究
- 产品规划
- 用户需求
- 硬件开发
- 采购和制造

Kyrozen可以补充他们缺少的角色。

4.4 早期创业者和小型团队

创业者可以使用Kyrozen完成：

- 产品机会分析
- MVP设计
- 技术路线选择
- 软件原型
- 硬件原型
- 早期测试
- 用户验证

减少早期组建完整团队的成本。

4.5 教师和教育机构

Kyrozen可以根据教学目标生成：

- 项目主题
 - 器材清单
 - 学习目标
 - 制作流程
 - 代码
 - 实验步骤
 - 测试标准
 - 教学材料
-

五、产品开发路线

Kyrozen不预设产品必须是软件或硬件。

在完成问题研究和产品定义后，Kyrozen根据实际需求选择合适路线。

5.1 纯软件路线

适用于：

- 网站
- 手机 App
- 桌面应用
- SaaS
- AI 工具
- 自动化工具
- 游戏
- 数据平台
- 开发者工具

基本流程：

问题发现
→ 市场研究
→ 产品定义
→ 用户体验设计
→ 软件架构
→ 开发
→ 测试
→ 部署
→ 用户验证
→ 迭代

5.2 硬件路线

适用于：

- Arduino项目
- ESP32设备
- 智能设备
- IoT产品

- 机器人
- 传感器设备
- 创客产品
- 电子原型

基本流程：

问题发现
→ 市场研究
→ 产品定义
→ 硬件架构
→ 元件选择
→ BOM和采购
→ 电路与固件开发
→ 组装
→ 硬件测试
→ 用户验证
→ 迭代

5.3 软硬件混合路线

适用于：

- 智能穿戴设备
- 智能家居
- 运动设备
- 机器人
- AI硬件
- 手机控制设备
- 云端IoT系统

Kyrosen需要同时规划：

- 设备硬件
 - 固件
 - 手机或网页界面
 - 云端服务
 - 数据传输
 - 用户账号
 - AI模型
 - 产品整体交互
-

5.4 非开发路线

有些问题不需要开发新产品。

研究后，Kyrozen可能建议：

- 使用已有软件
- 购买已有硬件
- 组合已有产品
- 改变工作流程
- 使用自动化服务
- 放弃项目

Kyrozen的目标是解决问题，而不是强迫用户开发产品。

六、完整产品开发流程

第一阶段：问题发现

Kyrozen首先帮助用户发现和理解问题，而不是立即进入开发。

用户可以：

- 描述自己的困扰
- 描述某类用户的困难
- 描述一个不满意的产品
- 上传图片或视频
- 描述一个工作流程
- 提出一个已有产品想法
- 让Kyrozen主动帮助寻找可解决的问题

Kyrozen需要识别：

- 谁遇到了问题
- 问题发生在什么场景
- 问题出现频率
- 问题造成什么影响
- 用户当前如何解决
- 当前方式有什么不足
- 用户真正需要的结果是什么

输出：

Problem Brief

主要包括：

- 目标用户
 - 使用场景
 - 表面问题
 - 深层需求
 - 当前解决方案
 - 尚未验证的假设
 - 初步机会方向
-

第二阶段：问题验证和市场调研

Kyrozen通过网络搜索和资料分析判断问题是否值得解决。

调研内容包括：

- 已有产品
- 竞争公司
- 软件和App
- GitHub开源项目
- Arduino和DIY项目
- 众筹项目
- 学术论文
- 专利
- 用户评论
- 社区讨论
- 失败或停止销售的产品
- 替代解决方式

Kyrozen不仅要回答“有没有类似产品”，还要分析：

- 这些产品具体解决了什么
- 为什么用户购买
- 用户主要抱怨什么
- 为什么有些产品失败
- 现有方案是否已经足够好
- 新方案可以在哪些方面不同
- 是否存在专利或技术风险

输出：

Market Research Report

包括：

- 市场现状
- 目标用户证据
- 竞品列表
- 功能对比
- 技术路线
- 用户反馈
- 市场缺口
- 可行性判断
- 风险
- 结论和建议

最终判断可以是：

- 继续开发
- 缩小范围
- 修改目标用户
- 改变产品形式
- 使用软件而非硬件
- 使用硬件而非软件
- 暂停
- 放弃

第三阶段：产品定义

当问题被初步验证后，Kyrozen帮助用户确定产品到底应该是什么。

主要工作：

- 明确产品目标
- 定义核心用户
- 建立价值主张
- 设计主要使用流程
- 转化用户需求
- 确定核心功能
- 删除不必要功能
- 定义MVP
- 确定成功标准
- 明确第一版不做什么

例如，用户想开发一副包含AI、GPS、健康检测、主动降噪和实时翻译的智能耳机。

Kyrozen不会直接尝试实现所有功能，而是帮助用户确定：

第一版只验证“运动状态能否自动影响音乐体验”。

输出：

Product Brief和PRD

包括：

- 产品目标
- 目标用户
- 用户故事
- 核心功能
- 功能优先级
- MVP范围
- 非目标
- 成功标准
- 成本限制
- 项目风险

第四阶段：解决方案设计

Kyrozen不应只生成一个方案。

它应该提供多个不同方向，例如：

- 最低成本方案
- 最容易实现方案
- 最好体验方案
- 纯软件方案
- 硬件方案
- 软硬件结合方案
- 使用现有产品组合的方案

每种方案需要比较：

- 功能
- 成本
- 难度
- 开发时间
- 用户使用难度
- 可靠性
- 后续扩展

- 风险

用户和Kyrozen共同选择最终方案。

Kyrozen记录：

- 比较过哪些方案
 - 为什么选择当前方案
 - 为什么放弃其他方案
 - 当前决策是否可逆
-

七、软件产品开发能力

当项目进入软件路线时，Kyrozen承担软件产品团队职责。

7.1 产品与交互设计

Kyrozen可以协助：

- 用户流程
 - 页面结构
 - 信息架构
 - UI方向
 - 原型
 - 交互逻辑
 - 可访问性
 - 首次使用流程
-

7.2 技术规划

Kyrozen根据项目需要选择：

- 前端框架
- 后端框架
- 数据库
- API结构
- 部署方案
- 认证系统
- AI模型
- 第三方服务

技术选择应基于实际需求，而不是默认使用最流行的技术。

7.3 软件开发

Kyrozen可以调用编程能力完成：

- 创建项目
- 编写代码
- 修改代码
- 管理文件
- 管理Git
- 运行测试
- 修复错误
- 优化代码
- 生成文档

OpenKyrozen现有的文件、终端和Git能力可以作为基础。

7.4 发布和运营

未来Kyrozen可以进一步支持：

- 网站部署
 - 云服务部署
 - 域名配置
 - App打包
 - 错误监控
 - 用户数据分析
 - 产品反馈管理
 - 版本规划
-

八、硬件产品开发能力

8.1 硬件架构

Kyrozen根据产品需求选择：

- 主控制器
- 传感器
- 显示器
- 执行器
- 通信方式
- 电源

- 存储
- 输入和输出设备

第一阶段主要支持：

- Arduino
 - ESP32
 - Raspberry Pi
 - 常见开发板
 - 常见传感器和模块
-

8.2 元件选择

Kyrozen不能只输出模糊元件名称。

应尽量明确：

- 制造商
- 芯片型号
- 模块型号
- 具体版本
- 数量
- 选择原因
- 兼容性
- 替代型号
- 注意事项

例如，不应只写：

ESP32

而应该明确到具体开发板型号和版本。

8.3 BOM和购买

Kyrozen为用户生成完整BOM，包括：

- 元件名称
- 精确型号
- 数量
- 用途
- 价格
- 购买链接

- 商家
- 库存
- 替代品
- 总成本

购买来源可包括：

- Amazon
- 淘宝
- 京东
- AliExpress
- DigiKey
- Mouser
- 官方商店
- 其他地区供应商

Kyrozen需要根据用户地区、预算和时间选择采购方案。

未来可支持：

- 一键加入购物车
- 按商家整理购物清单
- 价格对比
- 自动检查库存
- 用户标记已有元件

任何联盟购买链接都必须明确标注，不能影响推荐质量。

8.4 电路和接线

Kyrozen可以生成：

- 系统框图
- 接线图
- 引脚分配
- 电源连接
- 元件说明
- 分步骤组装指导

同时检查：

- 电压
- 电流
- GPIO冲突
- I²C地址
- 接口兼容

- 电源能力
 - 逻辑电平
 - 必要保护
-

8.5 固件开发

Kyrozen可以完成：

- Arduino代码
 - ESP32代码
 - PlatformIO项目
 - 库安装
 - 编译
 - 烧录
 - 串口读取
 - 错误修复
 - 传感器调试
 - 状态机设计
-

8.6 制作指导

对零基础用户，Kyrozen应提供专门的制作模式。

每次只显示一个步骤，例如：

步骤5：

将传感器的GND连接到ESP32的GND。

完成后拍一张照片。

用户可以上传：

- 照片
- 视频
- 串口日志
- 编译错误

Kyrozen根据这些信息帮助判断问题。

8.7 机械设计

未来可以增加：

- 外壳设计
 - 元件布局
 - 结构规划
 - 3D打印模型
 - CAD文件
 - 散热设计
 - 安装方式
 - 接口开孔
-

九、测试与产品验证

Kyrozen不把“代码成功运行”当作项目完成。

产品必须回到最初的问题进行验证。

软件测试

包括：

- 功能测试
- 界面测试
- 性能测试
- 安全测试
- 用户体验测试
- 多设备测试

硬件测试

包括：

- 编译测试
- 单个元件测试
- 集成测试
- 传感器测试
- 功耗测试
- 续航测试
- 稳定性测试
- 长时间运行测试

产品验证

Kyrozen帮助用户设计：

- 用户访谈
- 试用测试
- 反馈问卷
- 使用观察
- 对比测试
- 成功标准

最终回答：

产品是否真的改善了最初的问题？

测试结果将形成下一轮迭代计划。

十、Kyrozen的项目工作空间

Kyrozen不应只使用聊天记录保存项目。

每个项目应该有独立工作空间，包括：

- 当前项目状态
- 问题定义
- 调研资料
- 产品方案
- PRD
- 设计决策
- 软件代码
- 硬件方案
- BOM
- 测试结果
- 用户反馈
- 版本历史
- 下一步任务

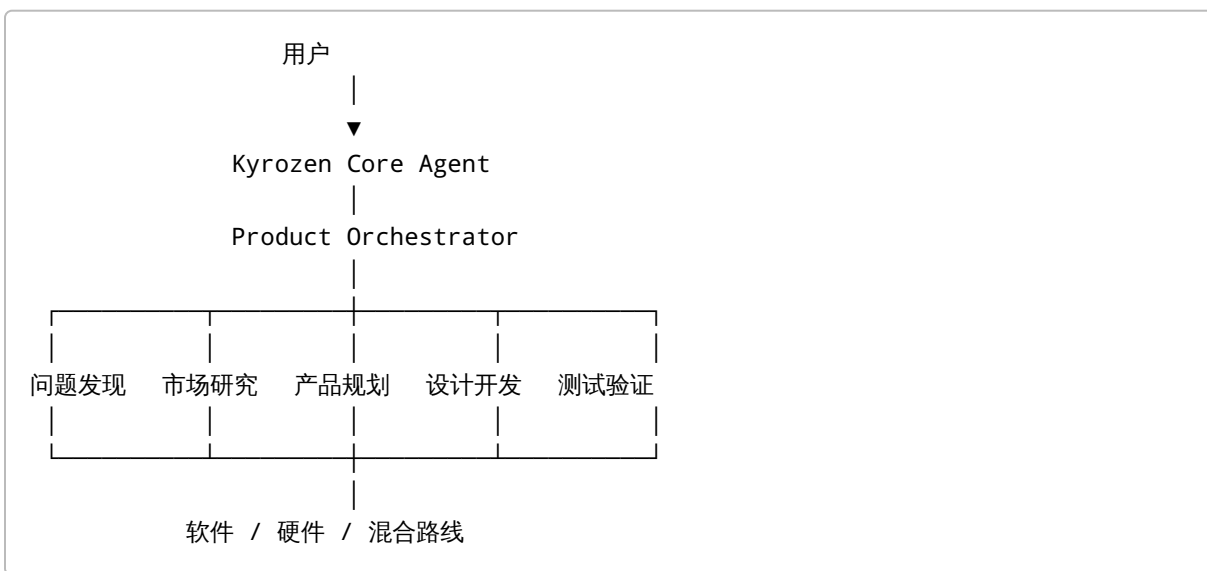
用户关闭项目后再次进入，Kyrozen仍然知道：

- 项目要解决什么问题
- 当前做到哪一步
- 已经做过什么决定
- 为什么做这些决定
- 遇到过哪些失败

- 下一步应该做什么

十一、Kyrozen Agent系统

Kyrozen采用一个中央产品Agent协调多个专业能力。



核心Agent

负责：

- 理解用户目标
- 管理项目阶段
- 调用专业能力
- 保存项目状态
- 管理任务
- 提醒风险
- 请求用户决策
- 检查项目是否偏离问题
- 决定下一步

专业能力

包括：

- Problem Discovery
- Market Research
- Product Management

- UX Design
- Software Engineering
- Hardware Engineering
- Procurement
- Firmware Development
- Mechanical Design
- Testing
- User Validation

这些能力可以由不同模型、提示词和工具实现，但在用户看来仍然是一个统一的Kyrozen。

十二、Kyrozen自学习系统

自学习是Kyrozen的重要特色，也是它与普通产品开发工具的主要区别之一。

Kyrozen不只被动等待用户下达任务。

它会围绕用户和项目持续学习、反思和寻找改进机会。

12.1 用户学习

Kyrozen逐渐了解：

- 用户的技术能力
- 用户预算习惯
- 用户常用工具
- 用户偏好的开发方式
- 用户的设计风格
- 用户可用设备
- 用户所在地区
- 用户常购买的商家
- 用户希望学习还是直接完成

这些信息会影响未来建议。

12.2 项目学习

Kyrozen记录：

- 项目目标
- 决策原因
- 已完成工作

- 测试结果
- 用户反馈
- 失败方案
- 成功方案
- 未解决风险

它不会因为开启新对话而失去项目上下文。

12.3 失败学习

Kyrozen会保存失败案例，例如：

问题：
ESP32无法控制LED。

原因：
使用了仅支持输入的GPIO。

解决：
更换引脚。

适用范围：
以后所有使用该ESP32型号的项目。

当未来出现类似问题时，Kyrozen优先参考已经验证的解决方法。

12.4 空闲反思

当用户没有操作时，Kyrozen可以在合理频率下分析项目，例如：

- 是否存在成本过高的元件
- 是否有更简单的实现方案
- 是否存在重复功能
- 是否遗漏测试
- 是否有未验证的假设
- 是否有新的竞品出现
- 是否存在新的技术或元件
- 软件架构是否过于复杂
- 硬件功耗是否超出目标
- 项目是否偏离最初问题

Kyrozen不会自动修改项目，而是生成改进建议。

例如：

当前硬件方案使用两个功能重叠的运动传感器。删除其中一个预计可降低18%的成本，并减少调试难度。

用户可以：

- 接受
- 忽略
- 延后
- 要求进一步研究

12.5 跨项目学习

在获得用户许可的情况下，Kyrozen可以将一个项目中验证过的经验用于其他项目。

例如：

- 常见ESP32引脚问题
- 某种传感器的噪声处理方案
- 某类App架构的成功经验
- 某类用户研究的有效方法
- 某供应商的型号差异

需要区分：

- 用户私有项目经验
- 可跨项目复用经验
- 公共产品开发知识

12.6 自学习控制

用户应当能够：

- 查看Kyrozen学到了什么
- 删除错误记忆
- 禁止某类学习
- 关闭空闲分析
- 限制项目间信息共享
- 将某些项目设为私密
- 回滚错误建议

自学习必须可见、可控、可纠正。

十三、核心产品界面

Kyrozen不应只是一个聊天框。

项目主页

显示：

- 项目名称
- 要解决的问题
- 当前阶段
- 当前进度
- 下一步
- 主要风险
- 当前预算
- 最近建议

对话界面

用户通过自然语言与Kyrozen交流。

项目画布

显示：

- 问题定义
- 市场调研
- 产品规划
- 产品设计
- 软件
- 硬件
- 采购
- 测试
- 迭代

决策中心

集中展示需要用户决定的事项。

采购中心

管理：

- 必买元件
- 可选元件
- 已有元件
- 已购买
- 已到货
- 替代型号

制作模式

面向手机和平板，逐步指导用户组装实体产品。

改进建议

显示Kyrozen在空闲分析中发现的：

- 风险
- 新机会
- 成本优化
- 架构优化
- 用户需求变化

十四、项目初期范围

Kyrozen长期可以支持各种产品，但第一版必须控制范围。

软件支持

第一版重点支持：

- 网站
- Web App
- 简单SaaS
- AI工具
- 自动化工具
- 简单桌面应用

硬件支持

第一版重点支持：

- Arduino
- ESP32
- Raspberry Pi
- 传感器
- LED
- 显示器
- 按钮
- 蜂鸣器
- 简单音频
- 小型舵机
- Wi-Fi和BLE设备

混合项目

支持较简单的：

- 设备 + Web控制台
- 设备 + 手机网页
- 设备 + 云端数据
- ESP32 + AI API

第一版不应支持：

- 高压电路
- 医疗设备
- 生命安全系统
- 自制高容量电池
- 工业安全设备
- 大型机器人
- 自动驾驶控制
- 需要复杂认证的量产产品

十五、第一版核心闭环

Kyrozen第一版必须证明：

一个没有专业背景的人，可以从一个问题开始，完成一个真实的软件或硬件产品原型。

第一版核心流程：

输入问题或想法

↓

问题分析

↓

基础市场调研

↓

产品定义

↓

选择软件、硬件或混合路线

↓

形成MVP

↓

完成开发方案

↓

AI执行开发

↓

测试

↓

用户验证

↓

生成改进建议

对于硬件项目额外包括：

具体BOM

↓

购买链接

↓

接线方案

↓

固件

↓

编译和烧录

↓

组装指导

↓

真实设备调试

十六、产品发展规划

第一阶段：产品原型

目标：

跑通完整流程。

主要功能：

- 问题发现
 - 市场调研
 - 产品定义
 - 软件和硬件路线判断
 - MVP规划
 - 软件开发
 - Arduino/ESP32开发
 - 项目记忆
 - 基础自学习
-

第二阶段：正式Beta

增加：

- 正式项目工作空间
 - 用户账户
 - 多项目
 - 采购中心
 - Local Hardware Bridge
 - 图片接线检查
 - 更完整的测试系统
 - 主动项目分析
 - 项目导出
-

第三阶段：设计和制造扩展

增加：

- PCB设计
- CAD
- 3D打印
- 电路仿真

- 软件自动部署
 - 手机App开发
 - 小批量制造建议
 - 供应链支持
-

第四阶段：完整产品开发平台

支持：

- 团队协作
 - 企业项目
 - 多Agent协作
 - 项目权限
 - 供应链管理
 - 制造服务
 - 产品运营
 - 用户反馈自动分析
 - 长期产品管理
-

十七、商业模式

免费版

- 基础问题发现
- 基础市场研究
- 单个项目
- 基础产品规划
- 有限软件或硬件生成

Pro版

- 多项目
- 完整市场研究
- 高级模型
- 完整软件开发
- 硬件采购
- 自动编译和调试
- 空闲项目分析
- 图片检查
- 项目导出

Education版

- 教师工作台
- 班级项目
- 学生进度
- 教学器材包
- 安全限制
- 教学文档

Team版

- 团队协作
- 项目权限
- Git集成
- 私有知识库
- 决策管理
- 自定义 workflow

其他收入

- 透明的采购联盟佣金
- 电子元件套件
- 教育项目套件
- PCB和3D打印服务
- 制造服务转介

十八、项目特色与竞争优势

从问题发现开始

多数AI开发工具从用户已经确定的产品需求开始。

Kyrozen可以从一个模糊问题开始。

软件和硬件统一

Kyrozen不预设产品形式，而是判断最适合解决问题的方法。

面向普通人

专业工具通常假设用户懂产品或工程。

Kyrozen的目标用户可以完全不懂技术。

完整闭环

Kyrozen覆盖：

问题
→ 调研
→ 产品
→ 开发
→ 制作
→ 测试
→ 改进

具体采购能力

硬件方案能够落实到：

- 具体型号
- 价格
- 购买链接
- 兼容性
- 替代品

长期项目记忆

Kyrozen持续理解项目，而不是每次重新开始。

主动自学习

Kyrozen在空闲时主动分析：

- 项目风险
- 改进机会
- 技术变化
- 成本变化
- 用户需求

继承OpenKyrozen

Kyrozen不需要重新从零构建通用Agent能力，而是可以继承OpenKyrozen已有的：

- 多模型

- 工具
 - Git
 - 文件
 - 终端
 - Web
 - 记忆
 - 插件
 - 自学习
-

十九、主要风险

项目范围过大

软件、硬件、市场和产品开发范围非常广。

解决方法：

第一版只支持有限类型的产品，并优先保证端到端成功率。

AI产生错误工程信息

硬件错误可能造成元件损坏。

解决方法：

- 使用可靠数据来源
- 明确置信度
- 自动兼容性检查
- 对高风险动作要求确认
- 限制第一版硬件范围

市场调研不准确

搜索结果可能过时或不完整。

解决方法：

- 保存来源
- 标记时间
- 区分事实和推断
- 显示证据强度
- 允许用户检查来源

自学习产生错误记忆

Kyrozen可能从失败或误解中学习错误结论。

解决方法：

- 用户可以查看记忆
- 删除错误学习
- 标记已验证和未验证经验
- 允许回滚
- 重要知识需要多次验证

用户对AI过度依赖

Kyrozen不能声称所有方案都绝对正确。

涉及安全、法律、医疗和量产的项目必须提醒用户寻求专业人员验证。

二十、项目成功标准

Kyrozen的成功不能只看生成了多少代码或文档。

真正的核心指标是：

用户是否完成了一个真实、有用的产品。

主要指标包括：

- 完成问题定义的项目比例
- 完成市场研究的项目比例
- 完成MVP定义的项目比例
- 软件原型成功运行率
- 硬件代码编译成功率
- 用户正确购买元件的比例
- 硬件原型最终运行率
- 测试失败后的修复成功率
- 用户验证后继续使用的比例
- Kyrozen主动建议被采纳的比例
- 用户完成第二个项目的比例

最终北极星指标：

没有相关专业背景的用户，在Kyrozen帮助下成功完成真实产品原型的比例。

二十一、最终定义

Kyrozen不是一个代码生成器，也不是一个只会讨论产品的聊天机器人。

它是：

一个能够理解问题、研究市场、规划产品、开发软件、设计硬件、帮助采购、指导制作、完成测试并持续学习改进的AI产品开发Agent。

OpenKyrozen为它提供Agent基础能力。

Kyrozen则将这些能力转化为一套完整、可持续、面向现实成果的产品开发系统。

它连接的是：

“我发现了一个问题。”

和：

“我做出了一个真正能够解决它的产品。”

之间的全部过程。